

Python — All Methods WITH Their Possible Errors

For every method: what it does, and the exact error it throws (and when). "—" means that method does not normally raise an error in typical use.

STRING METHODS

Method	Error it can raise	When
<code>capitalize()</code>	—	never errors
<code>casefold()</code>	—	never errors
<code>center(width, fillchar)</code>	<code>TypeError</code>	if fillchar is more than 1 character
<code>count()</code>	—	never errors, returns 0 if not found
<code>encode()</code>	<code>UnicodeEncodeError</code>	if char can't be encoded in given encoding
<code>endswith()</code>	<code>TypeError</code>	if argument isn't a string
<code>find()</code>	—	returns -1 if not found, never errors
<code>format()</code>	<code>IndexError</code> / <code>KeyError</code>	if placeholder index/key doesn't exist in args
<code>index()</code>	<code>ValueError</code>	substring not found
<code>is*()</code> (isalnum, isdigit, etc.)	—	always returns True/False, never errors
<code>join(iterable)</code>	<code>TypeError</code>	if any item in iterable isn't a string
<code>ljust()</code> / <code>rjust()</code> / <code>center()</code>	<code>TypeError</code>	if fillchar length != 1
<code>lower()</code> / <code>upper()</code> / <code>title()</code> / <code>swapcase()</code>	—	never errors
<code>lstrip()</code> / <code>rstrip()</code> / <code>strip()</code>	—	never errors
<code>partition()</code> / <code>rpartition()</code>	<code>TypeError</code>	if separator isn't a string
<code>replace()</code>	—	never errors, returns unchanged string if old not found
<code>rfind()</code>	—	returns -1 if not found
<code>rindex()</code>	<code>ValueError</code>	substring not found

Method	Error it can raise	When
<code>split()</code> / <code>rsplit()</code>	<code>ValueError</code>	if separator is an empty string <code>''</code>
<code>splitlines()</code>	—	never errors
<code>startswith()</code>	<code>TypeError</code>	if argument isn't a string
<code>translate()</code>	—	rarely errors
<code>zfill()</code>	—	never errors
Indexing <code>s[i]</code>	<code>IndexError</code>	position doesn't exist
Item assignment <code>s[0]='x'</code>	<code>TypeError</code>	strings are immutable
Concatenation <code>s + 5</code>	<code>TypeError</code>	can't concatenate str and int

LIST METHODS

Method	Error it can raise	When
<code>append(x)</code>	—	never errors
<code>clear()</code>	—	never errors
<code>copy()</code>	—	never errors
<code>count(x)</code>	—	never errors, returns 0 if absent
<code>extend(iterable)</code>	<code>TypeError</code>	if argument is not iterable
<code>index(x)</code>	<code>ValueError</code>	value not in list
<code>insert(i, x)</code>	—	never errors (out-of-range index just appends/prepends)
<code>pop(i)</code>	<code>IndexError</code>	index out of range, or empty list with default <code>pop()</code>
<code>remove(x)</code>	<code>ValueError</code>	value not in list
<code>reverse()</code>	—	never errors
<code>sort()</code>	<code>TypeError</code>	comparing incompatible types (e.g., str vs int)
Indexing <code>l[i]</code>	<code>IndexError</code>	position doesn't exist
<code>l[i] = x</code>	<code>IndexError</code>	index doesn't exist
<code>l + "text"</code>	<code>TypeError</code>	can only concatenate list with list
<code>l * "x"</code>	<code>TypeError</code>	can only multiply list by int

TUPLE METHODS

Method	Error it can raise	When
<code>count(x)</code>	—	never errors
<code>index(x)</code>	<code>ValueError</code>	value not in tuple
Indexing <code>t[i]</code>	<code>IndexError</code>	position doesn't exist
<code>t[i] = x</code>	<code>TypeError</code>	tuples are immutable, no item assignment
<code>t.append(x)</code> / any list-only method	<code>AttributeError</code>	tuple has no such attribute (only count/index exist)

DICTIONARY METHODS

Method	Error it can raise	When
<code>clear()</code>	—	never errors
<code>copy()</code>	—	never errors
<code>fromkeys()</code>	—	never errors
<code>get(key, default)</code>	<code>TypeError</code>	if more than 2 arguments passed
<code>items()</code> / <code>keys()</code> / <code>values()</code>	—	never errors
<code>pop(key)</code>	<code>KeyError</code>	key not found (and no default given)
<code>popitem()</code>	<code>KeyError</code>	dictionary is empty
<code>setdefault()</code>	—	never errors
<code>update()</code>	<code>TypeError</code>	if argument isn't a valid mapping/iterable of pairs
<code>d[key]</code>	<code>KeyError</code>	key doesn't exist
<code>d[unhashable] = x</code> (e.g. list as key)	<code>TypeError</code>	unhashable type

SET METHODS

Method	Error it can raise	When
<code>add(x)</code>	<code>TypeError</code>	if x is unhashable (list, dict, another set)
<code>clear()</code>	—	never errors
<code>copy()</code>	—	never errors
<code>difference()</code> / <code>intersection()</code> / <code>union()</code> / <code>symmetric_difference()</code>	<code>TypeError</code>	if argument isn't iterable
<code>difference_update()</code> / <code>intersection_update()</code> / <code>symmetric_difference_update()</code>	<code>TypeError</code>	if argument isn't iterable
<code>discard(x)</code>	—	never errors, silently does nothing if missing
<code>isdisjoint()</code> / <code>issubset()</code> / <code>issuperset()</code>	<code>TypeError</code>	if argument isn't iterable
<code>pop()</code>	<code>KeyError</code>	set is empty
<code>remove(x)</code>	<code>KeyError</code>	value not found
<code>update()</code>	<code>TypeError</code>	if argument isn't iterable
Indexing <code>s[0]</code>	<code>TypeError</code>	'set' object is not subscriptable

★ Master Error Pattern (applies across ALL types)

Error	Root Cause
<code>IndexError</code>	Position/index doesn't exist (str, list, tuple)
<code>KeyError</code>	Dict key or set element doesn't exist for that operation
<code>ValueError</code>	Right data type, but the value isn't present/valid
<code>TypeError</code>	Wrong data type used, wrong argument count, or unsupported operation between types
<code>AttributeError</code>	Calling a method that doesn't exist for that type (e.g. <code>tuple.append()</code>)
<code>NameError</code>	Using a variable name that was never defined or was deleted
<code>SyntaxError</code>	Code breaks Python's grammar rules (e.g. identifier starting with digit)

One-line master recall: *"Bad position → Index. Bad key → Key. Bad value → Value. Bad type/args → Type. Method doesn't exist for this type → Attribute."*